

DEVELOPER SPECIFICATION

Online Food Ordering App

Companion to the BRD — tech stack, schema, API reference, core logic, and a week-by-week build plan.

VERSION 1 · COD-ONLY, SINGLE RESTAURANT

Contents

01	Tech Stack
02	Project Structure
03	Environment Variables
04	Database Schema (Prisma)
05	API Reference
06	Core Business Logic
07	Angular App Structure
08	Development Plan
09	Testing Checklist
10	Deployment Checklist

01 • Tech Stack

Layer	Choice
Backend	Node.js + Express (modular monolith)
ORM	Prisma
Database	PostgreSQL — AWS RDS
Cache / ephemeral store	Redis — Render Key Value (OTP, refresh tokens, rate limits)
Frontend — Customer	Angular (standalone components), responsive/mobile-first
Frontend — Admin	Angular, separate app/build from the customer app
Image storage	Cloudinary
SMS (OTP + notifications)	MSG91
Hosting — app	Render.com (Web Service for API, two Static Sites for the Angular builds)
Hosting — DB	AWS RDS (PostgreSQL)
Auth	Phone + OTP (customer), username/password (admin) — JWT access + refresh tokens
Payment	Cash on Delivery only in v1

02 · Project Structure

Three repos (or three folders in a monorepo): backend/, customer-app/, admin-app/.

2.1 Backend

```
backend/
├── prisma/
│   └── schema.prisma
├── src/
│   ├── modules/
│   │   ├── auth/
│   │   │   ├── customer-auth.routes.ts    (send-otp, verify-otp, refresh, logout)
│   │   │   ├── admin-auth.routes.ts      (login, logout)
│   │   │   └── auth.service.ts
│   │   ├── menu/
│   │   │   ├── public-menu.routes.ts     (customer-facing, read-only)
│   │   │   └── menu.service.ts
│   │   ├── customer/
│   │   │   ├── profile.routes.ts
│   │   │   ├── addresses.routes.ts
│   │   │   └── orders.routes.ts
│   │   └── admin/
│   │       ├── menu-items.routes.ts
│   │       ├── categories.routes.ts
│   │       ├── orders.routes.ts
│   │       ├── settings.routes.ts
│   │       ├── dashboard.routes.ts
│   │       └── reports.routes.ts
│   ├── shared/
│   │   ├── middleware/
│   │   │   ├── authenticate.ts           (verifies JWT)
│   │   │   ├── requireRole.ts           (role guard)
│   │   │   └── rateLimiter.ts            (Redis-backed)
│   │   ├── utils/
│   │   │   ├── haversine.ts
│   │   │   ├── otp.ts                   (generate, hash, compare)
│   │   │   ├── cloudinary.ts
│   │   │   ├── redis/client.ts
│   │   │   └── prisma/client.ts
│   ├── app.ts                           (express app, route mounting)
│   └── server.ts                         (entry point)
├── .env.example
├── package.json
└── tsconfig.json
```

2.2 Customer App (Angular)

```
customer-app/src/app/
├── core/
│   ├── guards/auth.guard.ts
│   ├── interceptors/auth.interceptor.ts    (attaches JWT, handles refresh on 401)
│   └── services/ (auth, cart, menu, orders, addresses – one per domain)
├── features/
│   ├── login/
│   ├── otp-verify/
│   ├── profile-setup/
│   ├── home/
│   ├── item-detail/
│   ├── cart/
│   ├── addresses/
│   ├── checkout/
│   ├── order-confirmation/
│   ├── my-orders/
│   ├── order-detail/
│   └── profile/
└── app.routes.ts
```

2.3 Admin App (Angular)

```
admin-app/src/app/  
├─ core/  
│   ├── guards/admin-auth.guard.ts  
│   ├── interceptors/auth.interceptor.ts  
│   └── services/ (auth, menu, orders, settings, reports)  
├─ features/  
│   ├── login/  
│   ├── dashboard/  
│   ├── orders-queue/  
│   ├── order-detail/  
│   ├── menu-list/  
│   ├── menu-item-form/  
│   ├── categories/  
│   ├── settings/  
│   └── sales-report/  
└─ app.routes.ts
```

03 • Environment Variables

```
# Database
DATABASE_URL=postgresql://user:password@your-rds-endpoint:5432/foodapp

# Redis (Render Key Value)
REDIS_URL=redis://default:password@your-redis-host:6379

# JWT
JWT_ACCESS_SECRET=
JWT_REFRESH_SECRET=
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=30d

# MSG91 (SMS/OTP)
MSG91_AUTH_KEY=
MSG91_SENDER_ID=
MSG91_OTP_TEMPLATE_ID=

# Cloudinary
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=

# App
PORT=4000
NODE_ENV=production
CORS_ORIGIN_CUSTOMER=https://your-customer-app.onrender.com
CORS_ORIGIN_ADMIN=https://your-admin-app.onrender.com
```

Reminder: set these in Render's environment variable dashboard, not committed into a tracked `.env` file — see the BRD's Assumptions section for the reasoning.

04 • Database Schema (Prisma)

Ready to paste into `prisma/schema.prisma`. Run `npx prisma migrate dev --name init` to generate the migration.

```

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String      @id @default(uuid())
  phone       String      @unique
  name        String
  email       String?
  phoneVerifiedAt DateTime? @map("phone_verified_at")
  createdAt   DateTime     @default(now()) @map("created_at")
  addresses   Address[]
  orders      Order[]
  @@map("users")
}

model Admin {
  id          String      @id @default(uuid())
  username    String      @unique
  passwordHash String      @map("password_hash")
  createdAt   DateTime     @default(now()) @map("created_at")
  statusEdits OrderStatusHistory[]
  @@map("admins")
}

model Address {
  id          String      @id @default(uuid())
  userId      String      @map("user_id")
  user        User        @relation(fields: [userId], references: [id])
  addressText String      @map("address_text")
  latitude    Decimal      @db.Decimal(9, 6)
  longitude   Decimal      @db.Decimal(9, 6)
  label       String      @default("Home")
  isDefault   Boolean      @default(false) @map("is_default")
  createdAt   DateTime     @default(now()) @map("created_at")
  orders      Order[]
  @@map("addresses")
}

model Category {
  id          String      @id @default(uuid())
  name        String
  sortOrder   Int         @default(0) @map("sort_order")
  items       MenuItem[]
  @@map("categories")
}

model MenuItem {
  id          String      @id @default(uuid())
  categoryId   String      @map("category_id")
  category     Category    @relation(fields: [categoryId], references: [id])
  name         String
  description   String?
  price        Decimal      @db.Decimal(10, 2)
  imageUrl     String?      @map("image_url")
  cloudinaryPublicId String? @map("cloudinary_public_id")
  isAvailable   Boolean      @default(true) @map("is_available")
  createdAt     DateTime     @default(now()) @map("created_at")
  updatedAt     DateTime     @updatedAt @map("updated_at")
  orderItems    OrderItem[]
  @@map("menu_items")
}

enum OrderStatus {
  PLACED
  CONFIRMED
  PREPARING
  OUT_FOR_DELIVERY
  DELIVERED
  CANCELLED
}

model Order {
  id          String      @id @default(uuid())
  userId      String      @map("user_id")

```

```

user            User            @relation(fields: [userId], references: [id])
addressId       String?         @map("address_id")
address         Address?       @relation(fields: [addressId], references: [id])
deliveryAddressText String      @map("delivery_address_text")
deliveryLatitude Decimal        @db.Decimal(9, 6) @map("delivery_latitude")
deliveryLongitude Decimal       @db.Decimal(9, 6) @map("delivery_longitude")
status          OrderStatus    @default(PLACED)
totalAmount     Decimal         @db.Decimal(10, 2) @map("total_amount")
distanceKm      Decimal         @db.Decimal(6, 2) @map("distance_km")
isPaid          Boolean         @default(false) @map("is_paid")
createdAt       DateTime        @default(now()) @map("created_at")
cancelledAt     DateTime?       @map("cancelled_at")
items           OrderItem[]
statusHistory   OrderStatusHistory[]
@@map("orders")
}

model OrderItem {
  id          String    @id @default(uuid())
  orderId     String    @map("order_id")
  order       Order     @relation(fields: [orderId], references: [id])
  menuItemId  String    @map("menu_item_id")
  menuItem    MenuItem @relation(fields: [menuItemId], references: [id])
  itemNameSnapshot String @map("item_name_snapshot")
  priceSnapshot Decimal @db.Decimal(10, 2) @map("price_snapshot")
  quantity    Int
  @@map("order_items")
}

model OrderStatusHistory {
  id          String    @id @default(uuid())
  orderId     String    @map("order_id")
  order       Order     @relation(fields: [orderId], references: [id])
  adminId     String    @map("admin_id")
  admin       Admin     @relation(fields: [adminId], references: [id])
  status      OrderStatus
  changedAt   DateTime @default(now()) @map("changed_at")
  @@map("order_status_history")
}

model Settings {
  id          String    @id @default(uuid())
  shopLatitude Decimal   @db.Decimal(9, 6) @map("shop_latitude")
  shopLongitude Decimal   @db.Decimal(9, 6) @map("shop_longitude")
  deliveryRadiusKm Decimal @db.Decimal(5, 2) @default(2) @map("delivery_radius_km")
  minOrderValue Decimal   @db.Decimal(10, 2) @default(50) @map("min_order_value")
  isAcceptingOrders Boolean @default(true) @map("is_accepting_orders")
  updatedAt   DateTime @updatedAt @map("updated_at")
  @@map("settings")
}

```

05 · API Reference

5.1 Auth — Customer

Endpoint	Auth	Body	Response
POST /api/customer/auth/send-otp	None	{ phone }	{ success, expiresInSeconds: 300 }
POST /api/customer/auth/verify-otp	None	{ phone, otp }	{ accessToken, refreshToken, isNewUser }
POST /api/customer/auth/refresh	Refresh token	{ refreshToken }	{ accessToken }
POST /api/customer/auth/logout	Access token	{ refreshToken }	{ success }

5.2 Auth — Admin

Endpoint	Auth	Body	Response
POST /api/admin/auth/login	None	{ username, password }	{ accessToken, refreshToken }
POST /api/admin/auth/logout	Access token	{ refreshToken }	{ success }

5.3 Public Menu (no auth — browsable before login)

Endpoint	Auth	Response
GET /api/menu/categories	None	[{ id, name, sortOrder }]
GET /api/menu/items?categoryId=	None	[{ id, name, description, price, imageUrl, isAvailable, categoryId }]
GET /api/menu/items/:id	None	single item object
GET /api/settings/public	None	{ isAcceptingOrders, minOrderValue, deliveryRadiusKm }

5.4 Customer — Profile & Addresses

Endpoint	Auth	Body	Response
GET /api/customer/profile	customer	—	{ id, phone, name, email }
PATCH /api/customer/profile	customer	{ name?, email? }	updated profile
GET /api/customer/addresses	customer	—	[{ id, addressText, latitude, longitude, label, isDefault }]
POST /api/customer/addresses	customer	{ addressText, latitude, longitude, label, isDefault? }	created address, or 400 if outside delivery radius
PATCH /api/customer/addresses/:id	customer	same as above	updated address
DELETE /api/customer/addresses/:id	customer	—	{ success }

5.5 Customer — Orders

Endpoint	Auth	Body	Response

POST /api/customer/orders	customer	{ addressId, items: [{ menuItemId, quantity }] }	created order, or 400 with reason (below minimum, outside radius, shop closed, item unavailable)
GET /api/customer/orders?status=ongoing history	customer	—	list of orders
GET /api/customer/orders/:id	customer	—	order detail incl. status history
POST /api/customer/orders/:id/cancel	customer	—	{ success } or 400 if window has passed

5.6 Admin — Dashboard & Reports

Endpoint	Auth	Response
GET /api/admin/dashboard/summary	admin	{ todayOrders, pendingOrders, revenueToday, isAcceptingOrders }
GET /api/admin/reports/sales?from=&to=	admin	{ totalOrders, totalRevenue, dailyBreakdown: [...] }

5.7 Admin — Orders

Endpoint	Auth	Body	Response
GET /api/admin/orders?status=&since=	admin	—	list of orders (polled every 10s by the frontend)
GET /api/admin/orders/:id	admin	—	order detail
PATCH /api/admin/orders/:id/status	admin	{ status }	updated order; writes order_status_history row
PATCH /api/admin/orders/:id/payment	admin	{ isPaid: true }	updated order

5.8 Admin — Menu & Categories

Endpoint	Auth	Body	Response
GET /api/admin/menu/items	admin	—	all items incl. unavailable ones
POST /api/admin/menu/items	admin	multipart/form-data: name, categoryId, description, price, image	created item
PATCH /api/admin/menu/items/:id	admin	same fields, image optional	updated item
PATCH /api/admin/menu/items/:id/availability	admin	{ isAvailable }	updated item
DELETE /api/admin/menu/items/:id	admin	—	{ success } — also deletes Cloudinary asset
GET /api/admin/menu/categories	admin	—	categories list
POST /api/admin/menu/categories	admin	{ name, sortOrder? }	created category
PATCH /api/admin/menu/categories/:id	admin	{ name?, sortOrder? }	updated category
DELETE /api/admin/menu/categories/:id	admin	—	409 if items still reference it, else { success }

5.9 Admin — Settings

Endpoint	Auth	Body	Response
GET /api/admin/settings	admin	—	full settings row
PATCH /api/admin/settings	admin	{ shopLatitude?, shopLongitude?, deliveryRadiusKm?, minOrderValue?, isAcceptingOrders? }	updated settings

06 · Core Business Logic

6.1 Auth middleware & role guard

```
// shared/middleware/authenticate.ts
export function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Missing token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_ACCESS_SECRET);
    next();
  } catch {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}

// shared/middleware/requireRole.ts
export function requireRole(role: 'customer' | 'admin') {
  return (req, res, next) => {
    if (req.user.role !== role) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
}

// Mounting (app.ts)
app.use('/api/admin', authenticate, requireRole('admin'), adminRouter);
app.use('/api/customer', authenticate, requireRole('customer'), customerRouter);
app.use('/api/menu', publicMenuRouter); // no auth
app.use('/api/customer/auth', customerAuthRouter); // no auth (login itself)
app.use('/api/admin/auth', adminAuthRouter); // no auth (login itself)
```

6.2 OTP send + verify (Redis-backed)

```
// shared/utils/otp.ts
import crypto from 'crypto';
import bcrypt from 'bcrypt';

export function generateOtp(): string {
  return crypto.randomInt(100000, 999999).toString();
}

// POST /api/customer/auth/send-otp
async function sendOtp(req, res) {
  const { phone } = req.body;

  const rateLimitKey = `rateLimit:otp:${phone}`;
  const count = await redis.incr(rateLimitKey);
  if (count === 1) await redis.expire(rateLimitKey, 600); // 10 min window
  if (count > 3) return res.status(429).json({ error: 'Too many OTP requests, try again later' });

  const otp = generateOtp();
  const otpHash = await bcrypt.hash(otp, 8);
  await redis.set(`otp:${phone}`, JSON.stringify({ otpHash, attempts: 0 }), { EX: 300 });

  await sendSms(phone, `Your verification code is ${otp}. Valid for 5 minutes.`);
  res.json({ success: true, expiresInSeconds: 300 });
}

// POST /api/customer/auth/verify-otp
async function verifyOtp(req, res) {
  const { phone, otp } = req.body;
  const raw = await redis.get(`otp:${phone}`);
  if (!raw) return res.status(400).json({ error: 'OTP expired or not found' });

  const record = JSON.parse(raw);
  if (record.attempts >= 5) {
    await redis.del(`otp:${phone}`);
    return res.status(400).json({ error: 'Too many attempts, request a new OTP' });
  }

  const match = await bcrypt.compare(otp, record.otpHash);
  if (!match) {
    record.attempts += 1;
    await redis.set(`otp:${phone}`, JSON.stringify(record), { KEeptTL: true });
    return res.status(400).json({ error: 'Incorrect OTP' });
  }

  await redis.del(`otp:${phone}`);
  let user = await prisma.user.findUnique({ where: { phone } });
  const isNewUser = !user;
  if (!user) {
    user = await prisma.user.create({ data: { phone, name: '', phoneVerifiedAt: new Date() } });
  }

  const accessToken = signAccessToken({ id: user.id, role: 'customer' });
  const refreshToken = await issueRefreshToken(user.id); // stores refresh:{hash} -> userId in Redis, 30d TTL

  res.json({ accessToken, refreshToken, isNewUser });
}
```

6.3 Haversine distance check

```
// shared/utils/haversine.ts
export function haversineDistanceKm(lat1, lon1, lat2, lon2): number {
  const R = 6371;
  const dLat = (lat2 - lat1) * Math.PI / 180;
  const dLon = (lon2 - lon1) * Math.PI / 180;
  const a =
    Math.sin(dLat / 2) ** 2 +
    Math.cos(lat1 * Math.PI / 180) * Math.cos(lat2 * Math.PI / 180) * Math.sin(dLon / 2) ** 2;
  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
}

// Used both when saving an address AND again at order placement (defense in depth)
async function assertWithinDeliveryRadius(lat, lng) {
  const settings = await prisma.settings.findFirst();
  const distanceKm = haversineDistanceKm(settings.shopLatitude, settings.shopLongitude, lat, lng);
  if (distanceKm > settings.deliveryRadiusKm) {
    throw new BadRequestError(
      `Delivery not available beyond ${settings.deliveryRadiusKm}km. Your location is ${distanceKm.toFixed(1)}km away.`
    );
  }
  return distanceKm;
}
```

6.4 Order placement — full validation chain

```
// POST /api/customer/orders
async function placeOrder(req, res) {
  const { addressId, items } = req.body;
  const userId = req.user.id;

  const settings = await prisma.settings.findFirst();
  if (!settings.isAcceptingOrders) {
    return res.status(400).json({ error: 'Restaurant is currently not accepting orders' });
  }

  const address = await prisma.address.findFirst({ where: { id: addressId, userId } });
  if (!address) return res.status(404).json({ error: 'Address not found' });

  const distanceKm = await assertWithinDeliveryRadius(address.latitude, address.longitude);

  const menuItems = await prisma.menuItem.findMany({
    where: { id: { in: items.map(i => i.menuItemId) } }
  });
  for (const item of items) {
    const menuItem = menuItems.find(m => m.id === item.menuItemId);
    if (!menuItem || !menuItem.isAvailable) {
      return res.status(400).json({ error: `Item no longer available: ${menuItem?.name ?? item.menuItemId}` });
    }
  }

  const totalAmount = items.reduce((sum, item) => {
    const menuItem = menuItems.find(m => m.id === item.menuItemId);
    return sum + Number(menuItem.price) * item.quantity;
  }, 0);

  if (totalAmount < Number(settings.minOrderValue)) {
    return res.status(400).json({ error: `Minimum order value is ₹${settings.minOrderValue}` });
  }

  const order = await prisma.order.create({
    data: {
      userId,
      addressId,
      deliveryAddressText: address.addressText,
      deliveryLatitude: address.latitude,
      deliveryLongitude: address.longitude,
      distanceKm,
      totalAmount,
      status: 'PLACED',
      items: {
        create: items.map(item => {
          const menuItem = menuItems.find(m => m.id === item.menuItemId);
          return {
            menuItemId: menuItem.id,
            itemNameSnapshot: menuItem.name,
            priceSnapshot: menuItem.price,
            quantity: item.quantity
          };
        })
      }
    },
    include: { items: true }
  });

  await notifyOrderPlaced(order); // SMS via MSG91
  res.status(201).json(order);
}
```

6.5 Order cancellation — 60-second window

```
// POST /api/customer/orders/:id/cancel
async function cancelOrder(req, res) {
  const order = await prisma.order.findFirst({
    where: { id: req.params.id, userId: req.user.id }
  });
  if (!order) return res.status(404).json({ error: 'Order not found' });
  if (order.status !== 'PLACED') {
    return res.status(400).json({ error: 'Order can no longer be cancelled' });
  }
  const secondsElapsed = (Date.now() - order.createdAt.getTime()) / 1000;
  if (secondsElapsed > 60) {
    return res.status(400).json({ error: 'Cancellation window has passed' });
  }
  await prisma.order.update({
    where: { id: order.id },
    data: { status: 'CANCELLED', cancelledAt: new Date() }
  });
  res.json({ success: true });
}
```

6.6 Cloudinary — menu item image lifecycle

```
// shared/utils/cloudinary.ts
import { v2 as cloudinary } from 'cloudinary';
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET
});

export async function uploadMenuImage(fileBuffer: Buffer) {
  const result = await cloudinary.uploader.upload(
    `data:image/jpeg;base64,${fileBuffer.toString('base64')}`,
    { folder: 'menu-items' }
  );
  return { imageUrl: result.secure_url, publicId: result.public_id };
}

export async function deleteMenuImage(publicId: string) {
  if (publicId) await cloudinary.uploader.destroy(publicId);
}

// PATCH /api/admin/menu/items/:id — replacing an image
async function updateMenuItem(req, res) {
  const existing = await prisma.menuItem.findUnique({ where: { id: req.params.id } });
  let imageUrl = existing.imageUrl;
  let cloudinaryPublicId = existing.cloudinaryPublicId;

  if (req.file) {
    if (existing.cloudinaryPublicId) await deleteMenuImage(existing.cloudinaryPublicId);
    const uploaded = await uploadMenuImage(req.file.buffer);
    imageUrl = uploaded.imageUrl;
    cloudinaryPublicId = uploaded.publicId;
  }

  const updated = await prisma.menuItem.update({
    where: { id: req.params.id },
    data: { ...req.body, imageUrl, cloudinaryPublicId }
  });
  res.json(updated);
}
```

07 • Angular App Structure

7.1 Customer App Routes

```
// app.routes.ts
export const routes: Routes = [
  { path: 'login', component: LoginComponent },
  { path: 'verify-otp', component: OtpVerifyComponent },
  { path: 'welcome', component: ProfileSetupComponent, canActivate: [authGuard] },
  { path: '', component: HomeComponent }, // browsable without login
  { path: 'item/:id', component: ItemDetailComponent },
  { path: 'cart', component: CartComponent, canActivate: [authGuard] },
  { path: 'addresses', component: AddressListComponent, canActivate: [authGuard] },
  { path: 'addresses/new', component: AddEditAddressComponent, canActivate: [authGuard] },
  { path: 'checkout', component: CheckoutComponent, canActivate: [authGuard] },
  { path: 'order-confirmation/:id', component: OrderConfirmationComponent, canActivate: [authGuard] },
  { path: 'orders', component: MyOrdersComponent, canActivate: [authGuard] },
  { path: 'orders/:id', component: OrderDetailComponent, canActivate: [authGuard] },
  { path: 'profile', component: ProfileComponent, canActivate: [authGuard] },
];
```

7.2 Admin App Routes

```
// app.routes.ts
export const routes: Routes = [
  { path: 'login', component: AdminLoginComponent },
  { path: '', component: DashboardComponent, canActivate: [adminAuthGuard] },
  { path: 'orders', component: OrdersQueueComponent, canActivate: [adminAuthGuard] },
  { path: 'orders/:id', component: OrderDetailComponent, canActivate: [adminAuthGuard] },
  { path: 'menu', component: MenuListComponent, canActivate: [adminAuthGuard] },
  { path: 'menu/new', component: MenuItemFormComponent, canActivate: [adminAuthGuard] },
  { path: 'menu/:id/edit', component: MenuItemFormComponent, canActivate: [adminAuthGuard] },
  { path: 'categories', component: CategoriesComponent, canActivate: [adminAuthGuard] },
  { path: 'settings', component: SettingsComponent, canActivate: [adminAuthGuard] },
  { path: 'reports', component: SalesReportComponent, canActivate: [adminAuthGuard] },
];
```

7.3 Auth guard (shared pattern, both apps)

```
// core/guards/auth.guard.ts
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (auth.isValidSession()) return true;
  router.navigate(['/login']);
  return false;
};
```

7.4 Auth interceptor — attach token, refresh on 401

```
// core/interceptors/auth.interceptor.ts
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.getAccessToken();
  const authReq = token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req;

  return next(authReq).pipe(
    catchError(err => {
      if (err.status === 401) {
        return auth.refreshToken().pipe(
          switchMap(() => next(authReq.clone({ setHeaders: { Authorization: `Bearer ${auth.getAccessToken()}` } })))
        );
      }
      return throwError(() => err);
    })
  );
};
```

08 · Development Plan (6-8 weeks, solo developer)

Week 1

Foundations

Scaffold backend + both Angular apps; write Prisma schema and run initial migration; provision AWS RDS, Render Web Service, Render Key Value (Redis); configure environment variables.

Week 2

Auth

OTP send/verify + rate limiting, admin login, JWT access/refresh issuing, auth middleware + role guards on both sides, Angular login/OTP/profile-setup screens.

Week 3

Menu

Categories + menu item CRUD (admin), Cloudinary upload/delete wiring, public menu endpoints, Home/Item-Detail screens (customer), My Food List/Add-Edit Item screens (admin).

Week 4

Addresses & Settings

Address CRUD with map picker + "use current location," Haversine validation, Settings module (shop location, radius, min order, accepting-orders toggle) on the admin side.

Week 5

Orders — Customer side

Cart (client-side state), Checkout with full validation chain, order placement, 60-second cancellation, Order Confirmation and My Orders/Order Detail screens.

Week 6

Orders — Admin side

Orders Queue with 10-second polling, Order Detail with status transitions and payment marking, Dashboard summary, SMS notifications via MSG91 for order placed/status changes.

Week 7

Reports & polish

Sales Report screen, categories management UI, responsive QA pass across both apps, empty/error states, loading states.

Week 8

Testing & deployment

Run through the full testing checklist below, deploy all three services, do an end-to-end smoke test in production, fix what breaks. Buffer week if Week 1-7 overran.

09 • Testing Checklist

- OTP: correct code within 5 minutes succeeds; expired code fails; 6th wrong attempt forces a new OTP; 4th send within 10 minutes is rate-limited.
- Address radius: a point just inside the radius saves successfully; a point just outside is rejected with a clear message; changing the shop's radius in Settings doesn't alter distances already stored on past orders.
- Minimum order value: an order exactly at the minimum succeeds; one rupee below is rejected; changing the minimum in Settings takes effect immediately for new orders.
- Item availability race condition: an item is marked sold out by admin while a customer has it in cart — checkout must re-validate and reject that item, not just trust the client.
- Cancellation window: cancel succeeds at 59 seconds, fails at 61 seconds; cancelling an order that's already **CONFIRMED** is rejected regardless of elapsed time.
- Admin status transitions: verify each transition writes a row to `order_status_history` with the correct admin id.
- Payment marking: `isPaid` only settable by admin, never by the customer-facing API.
- Accepting-orders toggle: switching it off immediately blocks new order placement and shows the closed banner on the customer home screen.
- Route isolation (negative test): a valid customer JWT against any `/api/admin/*` route returns 403, and vice versa.
- Cloudinary lifecycle: replacing a menu item's image deletes the old asset; deleting a menu item deletes its asset too — no orphaned files.
- Refresh token flow: an expired access token triggers a silent refresh via the interceptor without logging the user out, until the refresh token itself expires or is revoked.

10 • Deployment Checklist

AWS RDS (PostgreSQL)

Create a PostgreSQL instance → note endpoint, port, username, password → set security group to allow inbound connections from Render's egress IPs (or public access with SSL required + strong password while confirming Render's static IP ranges) → confirm automated backups are enabled → run `npx prisma migrate deploy` against it once reachable.

Render — Backend Web Service

Connect the backend repo → set build command (`npm install && npx prisma generate`) and start command (`node src/server.js` or equivalent) → add all environment variables from Section 03 → deploy → confirm health check responds.

Render — Customer & Admin Static Sites

Two separate Static Site services, one per Angular app → build command `ng build --configuration production` → publish directory `dist/<app-name>` → set each app's API base URL to the backend Web Service's URL.

Render — Key Value (Redis)

Create the Key Value instance → copy its connection URL into the backend's `REDIS_URL` → confirm the backend connects on startup.

Cloudinary

Create account → note cloud name, API key, API secret → add to backend environment variables → do one test upload from the Add Menu Item screen to confirm the full round trip.

MSG91

Create account → complete DLT template registration for the OTP and order-status SMS templates (mandatory in India for transactional SMS) → note auth key and template IDs → add to backend environment variables → send a test OTP to your own number.

Final smoke test

From the live customer app: complete OTP login → browse menu → add an item → add an address just inside the radius → place a COD order → cancel within 60 seconds on a second test order. From the live admin app: log in → see the new order in the queue → progress its status → mark it paid → confirm Settings changes take effect on the live customer menu.